HashiCorp Certified: Terraform Associate（004）　目標 2d　#03

ステートのしくみ

ステートの 4 つの目的（対応づけ・メタデータ・性能・同期）、terraform.tfstate と JSON、refresh、show・state list・state show・state pull、lineage と serial、機密データ・リモートステート・ロック

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の State・Purpose of Terraform State・Remote State・State Storage and Locking・State Locking・Manage sensitive data・Backend Type: local・Command: show・state list・state pull・refresh）を書いた。英語の原文は『』で引用した

第1章

ステートの役割

設定と現実をつなぐ台帳。

ステートは設定と実物の対応表 『a necessary requirement for Terraform to function』

設定（.tf）
resource "aws_instance" "foo"

↕ ステート（terraform.tfstate）が対応づける

実物（AWS）
インスタンス ID i-abcd1234

> 『The primary purpose of Terraform state is to store bindings between objects in a remote system and resource instances declared in your configuration.』

なぜタグではだめ？	答え
初期の試作	ステートなしで AWS のタグを使っていた
問題	タグに対応しないリソース・クラウドがある
結論	Terraform 独自のステートで対応づける

> 1 つの実物は 1 つのリソースインスタンスだけに。import で 2 か所に取りこむと、対応があいまいになる。

出典：State／Purpose of Terraform State（Mapping to the Real World）

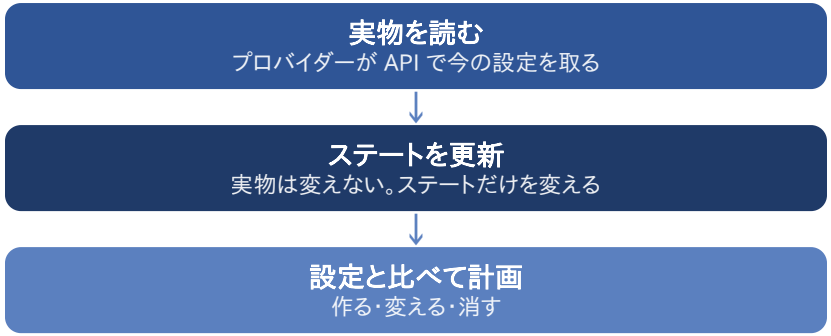
ステートの 4 つの目的

目的	中身	ないとどうなるか
対応づけ	設定のインスタンス ⇔ 実物の ID	どれを変える・消すのか分からない
メタデータ	最新の依存関係・最後に使ったプロバイダー設定（alias）	設定から消したリソースを消す順番が分からない
性能	全リソースの属性のキャッシュ（『the most optional feature』）	大きな環境で -refresh=false が使えない
同期	チームで同じステート → リモートステート＋ロック	各自が別のステートで、同時に実行してしまう

> 大きな環境：API は 1 件ずつで数百ミリ秒・回数制限もある → 『-refresh=false … -target』を使い、キャッシュしたステートを正とみなす。

出典：Purpose of Terraform State（Metadata／Performance／Syncing）

refresh：ステートを実物に合わせる



『Prior to any operation, Terraform does a refresh to update the state with the real infrastructure.』

書き方	扱い
plan・apply	自動で refresh する（ふつうはこれで足りる）
terraform refresh	非推奨。確認なしでステートを書きかえる
apply -refresh-only	変化を見てから承認できる（#13）

出典：State／Command: refresh

第2章

ステートの中身と見方

JSON だけど、手では書かない。

ステートファイルの中身

```
{
  "version": 4,
  "terraform_version": "1.12.0",
  "serial": 7,            # 書くたびに増える
  "lineage": "3f1c…",     # 作ったときの一意な ID
  "outputs": { … },
  "resources": [{
    "type": "aws_instance", "name": "foo",
    "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
    "instances": [{ "attributes": { "id": "i-abcd1234", … },
                    "dependencies": [ "aws_subnet.a" ] }]
  }]
}
```

『direct file editing of the state is discouraged』→ terraform state コマンドで。

形式は版で変わりうる。外部のソフトには show -json・output -json の出力を渡す。

state push は、lineage がちがう・行き先の serial が大きいと止まる（-force で押し切る前に state pull でバックアップ）。

出典：State（Inspection and Modification／Format）／State Storage and Locking（Manual State Pull/Push）

ステートを読むコマンド

コマンド	何が出るか	注意
terraform show	最新のステート（プランファイルを渡せばその計画）を人向けに	-json では sensitive も平文
terraform state list	リソースのアドレスの一覧（ルート → モジュールの順）	パターンで絞りこめる
terraform state show ADDR	1 つのリソースのくわしい属性	アドレスは state list で調べる
terraform output -json	ルートモジュールの出力値	自動化で apply の直後に保存
terraform state pull	ステートをそのまま標準出力に（jq と組み合わせ）	反対の push はとても危険

> plan・apply 以外でステートを変えると『lose track of managed resources』。変える前にバックアップ。

出典：Command: show／Command: state list／Inspect Terraform State／Command: state pull／Update Terraform state manually overview

第3章

守る・共有する

秘密が入る。だからリモートとロック。

ステートには秘密が入る

```
variable "db_password" {
  type      = string
  sensitive = true   # 表示は隠れる
}
resource "aws_db_instance" "main" {
  password = var.db_password
}
# → ステートには平文で残る
```

sensitive は『redacts those values from CLI output』だけ。ステートに残さないのは ephemeral・write-only（#10）。

Store your state remotely
リモートに置く

Encrypt your state at rest
HCP Terraform・S3（encrypt）・GCS

Use access controls
見られる人を限る

Use audit logs
だれが見たかを残す

ローカルでは平文のファイル → Git に入れない。

出典：Manage sensitive data（Background／State security best practices）

リモートステートとロック

ローカル（既定）	リモート
terraform.tfstate（local バックエンド）	HCP Terraform・S3・Azure Blob・GCS・Consul …
最新か・同時に動かしていないかを各自が確認	全員で共有。出力値をほかの設定へ読み取り専用で
ディスクに残る	失敗したとき以外ディスクに残さない

リモートでも console・state・taint などはローカルと同じように動く。

ステートのロック
書きこむおそれのある操作で自動。メッセージは出ない

ロックに失敗したら『Terraform does not continue』。対応しないバックエンドもある。

HCP Terraform は操作を中央のキューに並べ、承認待ちの計画があるのに新しい計画を作ろうとしたことも見つける。

local バックエンドも、システムの API でロックする。

出典：Remote State／State Storage and Locking／State Locking／Backend Type: local

本番での問われ方

ねらい	見るところ
ステートは何のため	対応づけがいちばんの目的。必須。タグでは代われない
メタデータと性能	消したリソースの削除順、キャッシュと -refresh=false
置き場所と形式	既定は terraform.tfstate、JSON、直接編集しない
読むコマンド	show・state list・state show・state pull のちがい
機密データ	平文で入る、sensitive は表示だけ、リモート＋暗号化、Git に入れない
チーム	リモートステートで共有、ロックは自動、失敗したら止まる

出典：Exam Content List（004）2d／本資料の問題演習（全 30 問）